

# Magic Square로 익히는 C2C × Dual-Track TDD

Cursor.AI · ARRR(Ask·Respond·Refine·Repeat) 실습

ASK

RESPOND

REFINE

REPEAT

4x4 마방진 하나로 주제 발견 → 설계 → 하네스 → ARRR을 처음부터 끝까지 따라 합니다

# 오늘 실습으로 만드는 것 · 7단계



## ① 주제 발견

Mom Test로 '진짜 문제' 한 문장



## ② 설계

R-G-I-O + validate\_lines 계약



## ③~⑥ ARRR

Ask→Respond→Refine→Repeat



## ⑦ 확장

새 기능 = 새 ARRR 사이클

③~⑥ = ARRR = C2C × Dual-Track TDD의 실행. 별개 단계가 아닙니다.

# 실습 환경 준비 (약 25분)



터미널에서 그대로 입력

```
mkdir MagicSquare_1004 && cd MagicSquare_1004  
cursor .
```

```
python -m venv .venv  
# Windows: source .venv/Scripts/activate  
# mac/Linux: source .venv/bin/activate
```

```
pip install pytest  
git init # GitHub 원격 연결
```

## 준비물

- 개인 PC
- GitHub 계정
- Cursor 로그인 계정
- Python 3.10+
- (선택) Node.js — MCP용

# 브랜치 전략 — 단계가 곧 브랜치

ARRR의 각 단계를 브랜치로 분리합니다. '어느 단계에서 무엇이 바뀌었나'가 이력만으로 추적됩니다.



```
git checkout -b staging      # main 에서 시작
git checkout -b spec        # 세션 3 - 설계·계약
git checkout -b red         # 세션 4 - A=Ask(RED)
git checkout -b green       # R=Respond(GREEN) → refactoring → new_features
```

## THE PROBLEM

# 관통 예제 — 4×4 마방진

|    |    |    |    |
|----|----|----|----|
| 16 | 2  | 3  | 13 |
| 5  | 11 | 10 | 8  |
| 9  | 7  | 0  | 12 |
| 4  | 14 | 15 | 0  |

빈칸(0) 정확히 2개 · 1~16 중복 없음

**마법상수 = 34**

모든 행·열·대각선 합이 34

**검증 대상 10선**

행 R1~R4 · 열 C1~C4 · 대각 D1(↘)·D2(↗)

**이번 실습 함수**

`validate_lines(grid)` → `status.failed_lines`

`find_blank_coords(grid)` → 빈칸 좌표

예: 위 격자 → [3,3,1,4,4,6] (작은 수→첫 빈칸, 좌표 1-index)

# C2C · Dual-Track · ARRR



## C2C

PRD → To-Do → Test Case → 구현  
연결고리를 끊지 않는 추적성



## Dual-Track(ECB)

Track A(UI/Boundary) +  
Track B(Domain/Logic)



## ARRR

Ask(RED)→Respond(GREEN)  
→Refine(REFACTOR)→Repeat

**ECB 방향: Entity는 Boundary/Control를 import하지 않는다 (순수 로직만).**

# Cursor 8계층 · Ask vs Agent

## Cursor.AI

|           |                    |
|-----------|--------------------|
| Model     | 생각하는 엔진            |
| Agent     | 파일 생성·실행 작업자       |
| Harness   | 실행 환경(pytest·venv) |
| Rule      | 헌법 (.cursorrules)  |
| Skill     | 긴 절차 매뉴얼           |
| Tool/MCP  | 터미널·외부 연동          |
| Command   | /슬래시 버튼            |
| Test Loop | 검증(pytest·Golden)  |

## Ask 모드

분석·리뷰만. 파일이 안 바뀜.  
"리뷰만" "맞는지 봐줘"

## Agent 모드

파일 생성·수정·pytest·git.  
"만들어줘" "실행해"

## 주제 발견 — Mom Test (전 과정 Ask)

### 3원칙

- 내 아이디어 말하지 않기
- 과거·구체·사실만 묻기
- 칭찬·미래 의견 무시

### 좋은 답변 vs 나쁜 답변

- ✓ “지난주 대각선 빼먹어 20분 날렸다”
- ✗ “자동 검증 앱 있으면 좋겠다” (솔루션)

MagicSquare Mom Test. 페르소나: 부분 마방진(빈칸 2개) 학습자.

Mom Test 규칙 준수. 질문 1개만. 솔루션 금지.

→ 답변 후: 사실성 평가 + 다음 추궁 1개 + 불편 요약



## 진짜 문제 → 주제 한 문장

표면 ✕

"4x4 마방진 만드는 프로그램을 만든다"

Mom Test ✔

"10선 합을 수동 확인하느라 시간·실수가 나는 문제를, 같은 기준으로 반복 판정하게 한다"

세션 3 주제

"유효 배치 판정·확보를 Cursor Rule/Command로 반복 가능하게 만든다"

진짜 문제엔 솔루션 이름(TDD/PyQt/Cursor)이 없어야 합니다. → 이 문장이 R-G-I-O의 출발점.

## 설계 — R-G-I-O & validate\_lines 계약

R

TDD·ECB 지키는 시니어 개발자

G

10선 합 34 즉시 판정 + 실패 줄 표시

I

4×4 격자 (0=빈칸, 1~16)

O

pass/fail/incomplete + 실패 줄 ID

성공 기준 = 테스트의 씨앗 → 여기서 C2C 시작

### validate\_lines 계약

**incomplete** 빈칸(0) 있음

**pass** 16칸 + 10선 모두 34

**fail** 34 아닌 줄 → failed\_lines

줄 ID: R1~R4·C1~C4·D1~D2

## 꼭 만드는 3가지 (Agent 모드)



### Harness

pyproject.toml + src/ + tests/



### Rule

.cursorrules (도메인·계약·TDD)



### Command

/tdd-red (RED 절차 버튼)

```
[tool.pytest.ini_options]
pythonpath = ["."] # 없으면 import 실패
testpaths = ["tests"]
```

# .cursorrules 핵심

마법상수 34 = SSOT (리터럴 산재 금지)

10선: R1~R4·C1~C4·D1·D2

RED→GREEN→REFACTOR, skip/xfail 금지

RED은 tests/만 수정, src/ 금지

## RED 확인 — FAIL이 정상

```
/tdd-red
```

T2: 완성 격자에서 (2,2)=6 을 7 로 바꿔 R2·C2 가 35가 되는 fail 케이스.  
기대: status="fail", failed\_lines 에 "R2"와 "C2" 포함. src/ 수정 금지.

```
$ pytest tests/test_validate_lines.py -v  
test_t2_row_sum_not_34_is_fail FAILED  
1 failed  
# validate_lines 미구현 → FAIL 이 정상
```



### RED의 의미

RED 성공 = FAIL. PASS가 나오면 src/에 구현이 미리 든 것 → src/ 비우고 다시.

# ARRR ↔ Command ↔ 모드

**A — Ask (RED)**

/red-test-plan → /red-skeleton

Ask / Agent

**R — Respond (GREEN)**

/green-minimal → /golden-master

Agent

**R — Refine (REFACTOR)**

/refactor-smell → /refactor-safe

Ask / Agent

**R — Repeat**

/export · /export-session

Agent

“Ask 단계에서 RED를 진행한다” — 각 단계마다 세션 3에서 만든 Command를 꺼내 씁니다.

A = ASK (RED)

## C2C 추적 & Dual-Track 설계표

PRD "빈칸에 유효 숫자 입력"  
→ To-Do "빈칸(0) 두 곳을 찾는다"  
→ TestCase find\_blank\_coords()  
→ RED 실패 확인 후 구현

C2C 3규칙: 판단 항목만 변환 · 1 To-Do:1 TC · RED 먼저

### Dual-Track 설계표

#### Track B (Logic) — tests/entity/

D-LOC-01 find\_blank\_coords()

G1 → [(2,3),(4,4)] (I6 row-major)

#### Track A (UI) — tests/boundary/

U-IN-01 grid=None → E003

Layer만 boundary로 바뀌 재사용

A = ASK · 실습

## RED 설계 → 스켈레톤 → FAIL

Ask · /red-test-plan

```
/red-test-plan
Phase: red | Layer: entity | Track: Logic
이번 RED 묶음: D-LOC-01 (FR-LOC-01)
설계표만. tests/.src/ 만들지 마.
```

Agent · /red-skeleton

```
/red-skeleton
Test ID: D-LOC-01
픽스처: tests/conftest.py (grid_g1)
pytest.fail 한 줄만. src/ 금지.
```

```
def test_d_loc_01_blank_coords_row_major(grid_g1):
    # Given: G1 격자(00| 2개)  When: find_blank_coords(grid_g1)
    # Then: [(2,3),(4,4)] (1-index, row-major)
    pytest.fail("RED: D-LOC-01 - 구현 없음, 의도적 실패")  # → 1 failed (RED 정상)
```

R = RESPOND (GREEN)

## 최소 구현 → PASS

```
/green-minimal  
Phase: green | Layer: entity | Track: Logic  
RED 대상: D-LOC-01  
최소 구현만. constants SSOT. 1커밋=1 RED.
```

```
# src/entity/find_blank_coords.py (최소)  
from .constants import BLANK_CELL  
def find_blank_coords(grid):  
    return [(r+1, c+1) # I6 1-index  
            for r,row in enumerate(grid)  
            for c,v in enumerate(row) if v==BLANK_CELL]
```

### GREEN 4단계

- 1 RED 재확인 (의도적 실패)
- 2 통과할 최소 코드만
- 3 pytest.fail → assert 교체 → PASS
- 4 git commit (1커밋=1 RED)

#### 금지

하드코딩·매직넘버 · 다른 ID 동시 해결 · REFACTOR ·  
assert 완화



# Golden Master — 회귀 안전망

GREEN PASS 후 '현재 출력'을 기준 파일로 저장 → 리팩터링에서 출력이 바뀌면 diff로 즉시 잡힘.

```
/golden-master 대상: D-LOC-01 (PASS 후)
1. tests/_approval.py 생성
2. tests/golden/d_loc_01...approved.txt 연결
3. UPDATE_GOLDEN=1 pytest ... # 기준 생성
4. UPDATE_GOLDEN 없이 pytest # matched 확인
```

```
# d_loc_01...approved.txt
2,3
4,4

# 1-index row,col
# row-major (I6)
```

⚠ 순서 주의 GREEN PASS → \_approval·golden 연결 → 기준 생성 → 검증. 연결 없이 UPDATE\_GOLDEN=1 은 무시됩니다.

R = REFINE (REFACTOR)

## 스멜 탐지 → 안전 정제

/refactor-smell (Ask, 수정 금지)  
전제: pytest 전부 PASS  
P0/P1/P2 스멜 표 → 후보 1~3개

/refactor-safe (Agent)  
스멜 1개(P0): Duplicated - 10선 합 4회  
→ `_collect_failed_line_ids()` 추출  
계약 불변 · Budget(파일≤3·메서드≤3)  
→ pytest + golden matched 확인

### Refine 안전 원칙

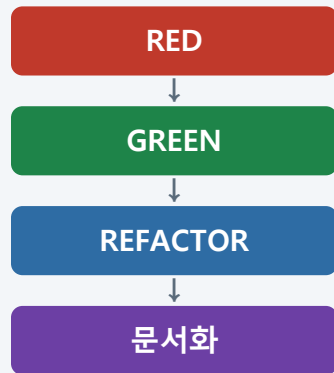
- 외부 계약(입출력·예외) 변경 금지
- 매 커밋 pytest PASS + GM matched
- Change Budget: 파일≤3·클래스≤1·메서드≤3
- 기능 추가·버그 수정은 리팩터 아님

R = REPEAT

## 문서화로 닫고, 다음 RED로

한 사이클을 Report/Transcript로 닫고 다음 Ask(RED)로. 테스트가 모두 통과할 때까지 최소 3회 반복.

```
/export                # Report/NN · Prompting/NN  
/export-session  
Phase: repeat | Track: Logic+UI  
ARRR 1사이클 보고 + 다음 RED 후보
```



↳ 최소 3회

문서화는 *magic-square-docs Skill*로 자동화 (번호 자동증가·형식 고정). 채팅에 없는 결과 기재 금지.

# C2C 추적성 매트릭스

코드 한 줄을 보고 '어느 Test ID·불변식을 지키나?'를 즉답할 수 있으면 C2C 완성.

| Test ID  | 계약(불변식)            | 검증 테스트                 | 구현                      |
|----------|--------------------|------------------------|-------------------------|
| D-LOC-01 | I6 빈칸 좌표 row-major | entity::test_d_loc_01  | find_blank_coords()     |
| D-MIS-01 | I7·I11 미존재 수 오름차순  | entity::test_d_mis_01  | find_not_exist_nums()   |
| D-VAL-01 | I1~I5 마방진 판정       | entity::test_d_val_01  | is_magic_square()       |
| U-IN-01  | E003 grid=None 차단  | boundary::test_u_in_01 | InputHandler.validate() |
| (판정)     | status 3종·10선 34   | test_validate_lines    | validate_lines()        |

## 자주 하는 실수 5가지

✗ RED인데 PASS가 나옴

src/에 구현이 미리 든 것 → src/ 비우고 다시 (RED 성공=FAIL)

✗ Ask로 리뷰시켰는데 파일이 바뀜

모드를 Ask로, 프롬프트 끝에 "수정하지 마"

✗ No module named 'src'

pyproject.toml 에 pythonpath = ["."]

✗ UPDATE\_GOLDEN=1 인데 PASS만

GREEN PASS → \_approval·golden 연결부터

✗ entity가 boundary를 import

ECB 위반 — entity는 순수 로직만

# 한 사이클 타임박스 & 체크



## ARRR 1사이클 (~45분)

|                    |     |
|--------------------|-----|
| Ask (RED)          | 15분 |
| Respond (GREEN)+GM | 15분 |
| Refine (REFACTOR)  | 10분 |
| Repeat (문서화)       | 5분  |

## 사이클 체크리스트

- RED: tests/만, FAIL 확인
- GREEN: 최소 구현 + PASS
- GM: matched (diff 없음)
- REFACTOR: 스멜 1개 + 회귀
- 커밋 분리 (test/feat/refactor)
- /export · GitHub 업로드

## 기능 확장 — 새 ARRR 사이클

1차 사이클이 끝나면 기능을 키웁니다. 새 기능 = 새 테스트(Ask) → 확장 → 재검증. 추적은 유지.



각 버전마다 테스트를 먼저 추가(Ask)하므로, 확장도 추적 가능한 상태(C2C)로 진행됩니다.

브랜치: new\_features

WRAP - UP

# 마방진 한 칸을 채우듯, 계약 하나에 테스트 하나, 구현 하나

*그 연결을 끝까지 끊지 않는 것 — 그것이 C2C입니다.*

## 세션 3

Mom Test · R-G-I-O · 하네스

## 세션 4

Ask→Respond→Refine→Repeat

## 추적성

Test ID↔테스트↔구현 매트릭스

Cursor.AI · Python 3.12 · pytest · Git | ARRR ≡ RED · GREEN · REFACTOR + Repeat